```
title: "SpringCloud"
date: 2023-11-11T15:08:23
tags: [ "微服务" ]
```

SpringCloud

> 前言：本次SpringCloud微服务学习涉及NetFlix和Alibaba两不同版本，主要对应关系为：

Spring Cloud Alibaba 套件和 Spring Cloud Netflix 套件类比

Spring Cloud Alibaba 套件和 Spring Cloud Netflix 套件大致的对应关系：

- Nacos = Eureka/Consule + Config + Admin
- Sentinel = Hystrix + Dashboard + Turbine
- Dubbo = Ribbon + Feign
- RocketMQ = RabbitMQ
- Schedulerx = Quartz

微服务原则：所有的服务都需要保证职责的单一原则

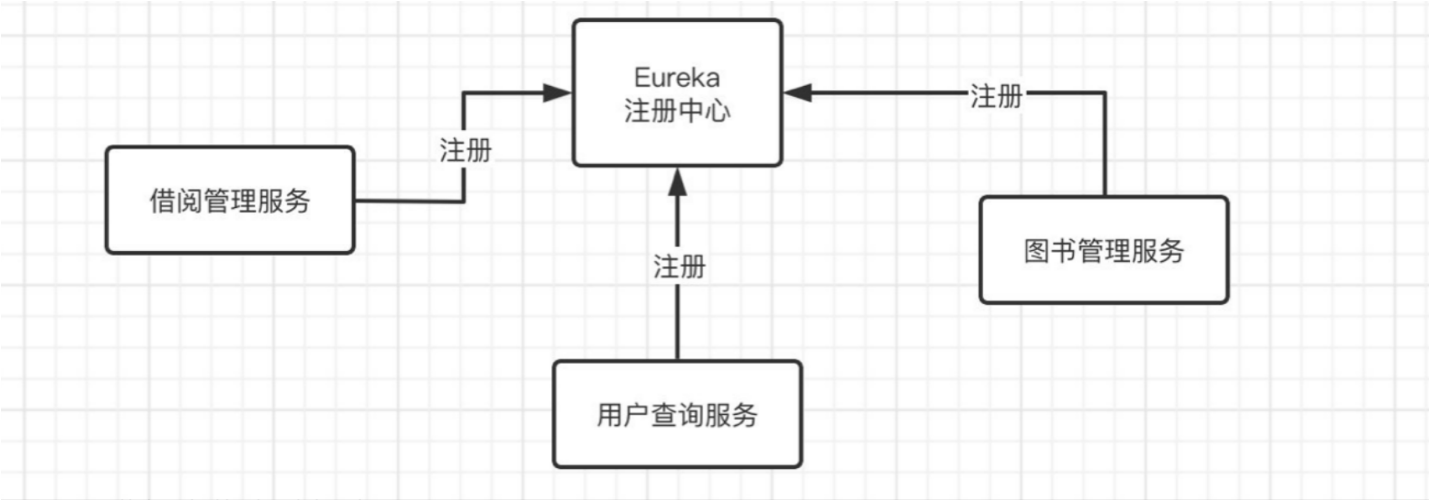
SpringCloud作为一系列框架的有序集合，是**SOA**的延续

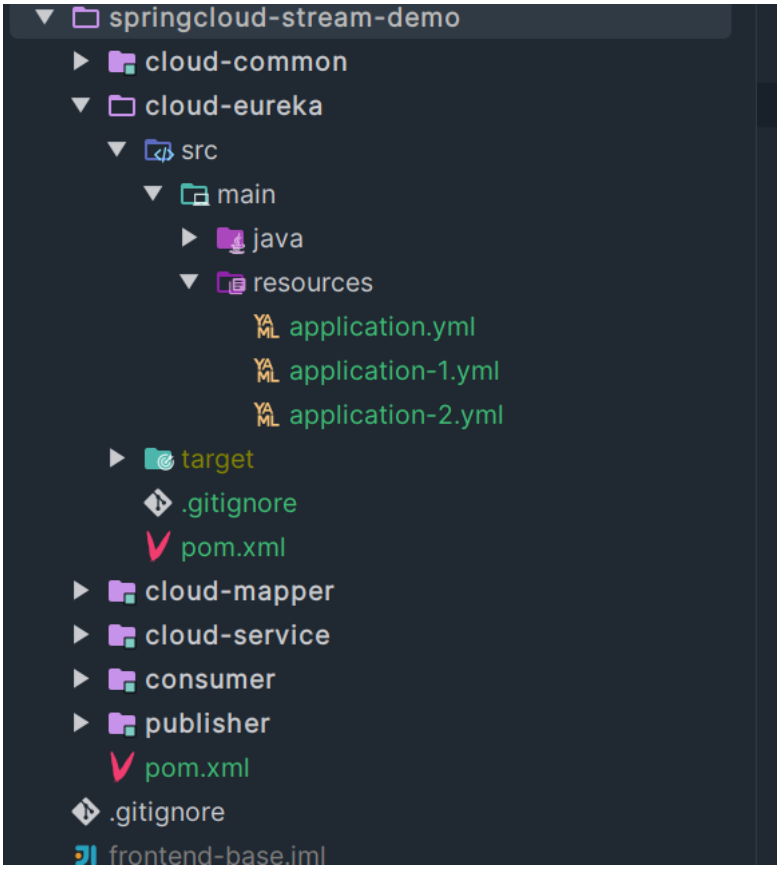
SpringCloud Alibaba 是遵循SpringCloud来开发的套件，Nacos亦然

服务注册和发现

Eureka

Eureka能够自动注册并发现微服务，然后对服务的状态、信息进行集中管理，这样当我们需要获取其他服务的信息时，我们只需要向Eureka进行查询就可以了。





> 通过Eureka服务器进行服务注册与发现，那么现在我们来看看，它的负载均衡到底是如何实现的，实际上之前演示的负载均衡是依靠LoadBalancer实现的。

> 在2020年前的SpringCloud版本是采用Ribbon作为负载均衡实现，但是2020年的版本之后SpringCloud把Ribbon移除了，进而用自己编写的LoadBalancer替代。

负载均衡

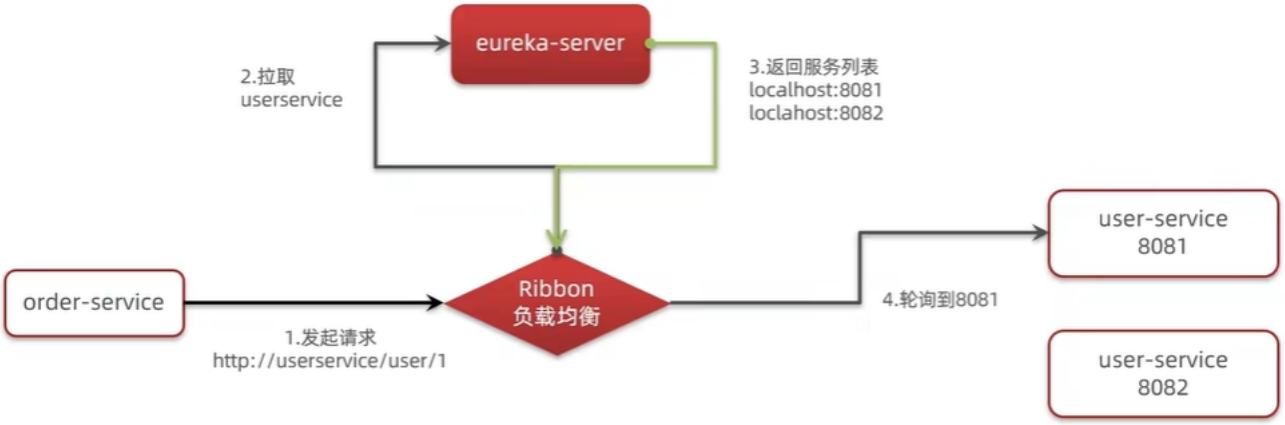
实际上，在添加`@LoadBalanced`注解之后，会启用拦截器对我们发起的服务调用请求进行拦截（注意这里是针对我们发起的请求进行拦截），叫做`LoadBalancerInterceptor`，它实现`ClientHttpRequestInterceptor`接口：

```java
@FunctionalInterface
public interface ClientHttpRequestInterceptor {
    ClientHttpResponse intercept(HttpRequest request, byte[] body, ClientHttpRequestExecution execut
}
```

如何找到服务？

Ribbon负载均衡

负载均衡流程



Nacos注册中心

功能：

- 服务注册发现
- 配置中心
- 服务元数据/实例元数据：可以用于流量控制（在实例的metadata（元数据）中加上标签信息。通过IRule获取Server列表并根据这些Server中元数据的标签信息决定路由情况）
- 局域网内服务信息传输快

服务集群分级模式

服务-集群-实例

修改yml：

```yml
spring:
  cloud:
    nacos:
      server-addr: localhost:8848
      discovery:
        cluster-name: ClUSTER-01
```

现在我们希望修改默认的负载均衡策略，可以进行指定，比如我们现在希望用户服务采用随机分配策略，我们需要先创建随机分配策略的配置类（不用加`@Configuration`）：

```java
public class LoadBalancerConfig {
        //将官方提供的 RandomLoadBalancer 注册为Bean
    @Bean
    public ReactorLoadBalancer<ServiceInstance> randomLoadBalancer(Environment environment, LoadBala
        String name = environment.getProperty(LoadBalancerClientFactory.PROPERTY_NAME);
        return new RandomLoadBalancer(loadBalancerClientFactory.getLazyProvider(name, ServiceInstanc
    }
}
```

对指定服务进行配置

必须重写负载均衡的轮询规则才可以改变负载均衡规则（实现方式为注入Bean或yml配置）

```
@Configuration
@LoadBalancerClient(value = "userservice",      //指定为 userservice 服务，只要是调用此服务都会使用我们指定的策略
                    configuration = LoadBalancerConfig.class)   //指定我们刚刚定义好的配置类
public class BeanConfig {
    @Bean
    @LoadBalanced
    RestTemplate template(){
        return new RestTemplate();
    }
}
```

这样子就实现了对服务的指定负载均衡策略配置

集群优先

Nacos优先访问本地集群，当本地集群宕机时，会发生跨集群访问，确定实例之后，会在本地实例中随机轮询

权重负载均衡

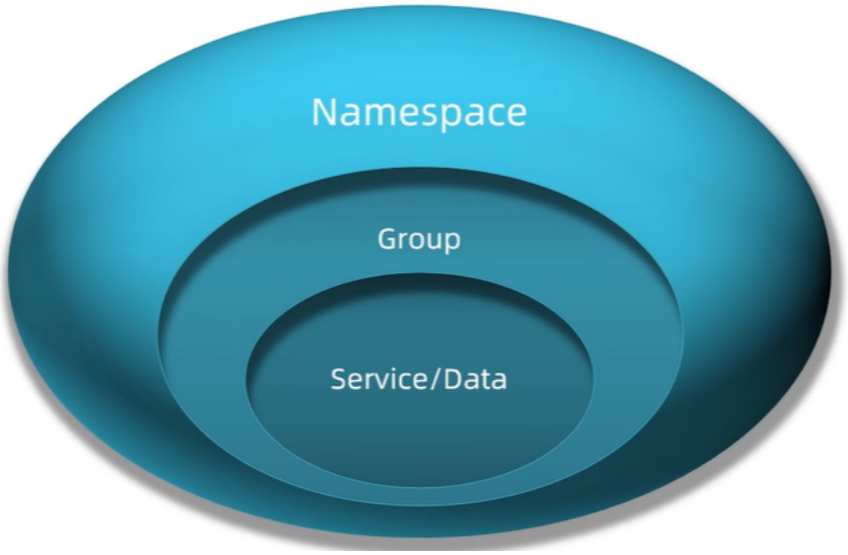
和Nginx差不多，通过更改集群中实例权重改变负载策略

利用：将服务实例权重调为0~0.01,放入少量请求实现服务升级

环境隔离

namespace

Nacos中服务存储和数据存储的最外层都是一个名为namespace的东西，用来做最外层隔离



相关度高时可以放入同一组

不同namespace之间不可通过RPC相互调用！

心跳检测

临时实例挂掉直接剔除，非临时实例（主动检测）挂掉时，Nacos将会询问健康状况，同时挂起直到实例重启，同时主动发送服务实例变更消息给服务调用方。Nacos采用CP，Eureka采用AP

Nacos配置管理

Data-ID：ServiceName-dev/pro.yaml

依赖：

```
<dependency>
    <groupId>com.alibaba.cloud</groupId>
    <artifactId>spring-cloud-starter-alibaba-nacos-config</artifactId>
</dependency>
```

一般只在有热更新需求的配置加入 Nacos配置中心

统一配置管理

添加bootstrap.yml

实现热更新

注解@Refreshed

配置文件根据不同环境，具有共享配置和dev/test配置文件等，配置优先级为远端>服务名>本地环境

微服务会从nacos读取的配置文件：

① [服务名]-[spring.profile.active].yaml，环境配置

② [服务名].yaml，默认配置，多环境共享

优先级：

① [服务名]-[环境].yaml >[服务名].yaml > 本地配置

负载均衡

Spring实现了一套自己的负载均衡LoadBalancer.
实际上在进行负载均衡的时候，会向Eureka发起请求，选择一个可用的对应服务，然后会返回此服务的主机地址等信息,然后才进行实际的服务调用。

负载均衡策略

- 轮询策略
- 权重策略
- 随机策略
- 最小连接数策略（选择连接数最小的的服务实例）
- 重试策略（当服务实例尝试失败之后，尝试继续获取，若多次失败返回null）
- 可用性敏感策略（随机轮询，判断是否可用）
- 区域敏感策略（可用和不可用相互的分区中选取可用分区中的实例）

限流方案

- Guava RateLimiter：基于令牌桶算法限流，单机的。
- Sentinel：基于滑动窗口、漏桶、令牌桶限流，支持单机，也支持集群。
- Redission RateLimiter：令牌桶，支持单机，也支持集群。

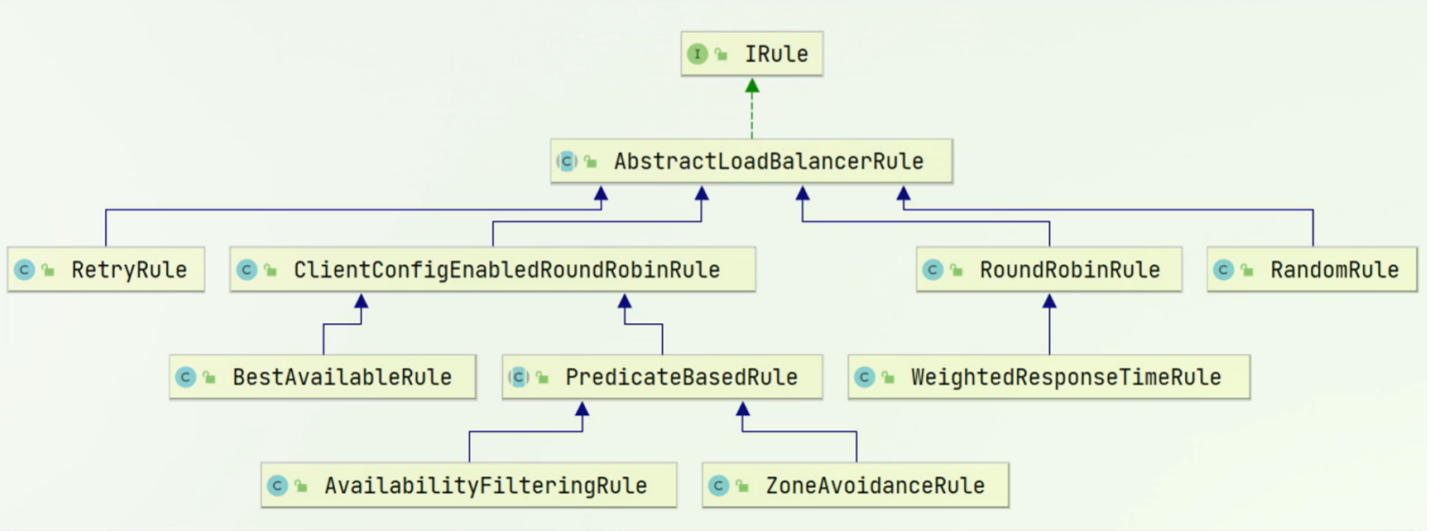
添加依赖：

```
<dependency>
                        <groupId>org.springframework.cloud</groupId>
                        <artifactId>spring-cloud-loadbalancer</artifactId>
                </dependency>
```

自定义负载均衡策略
LoadBalancer默认提供了两种负载均衡策略：

RandomLoadBalancer - 随机分配策略
(默认) RoundRobinLoadBalancer - 轮询分配策略

服务名称-》 LoadBalancerInterceptor-> execute -》 通过轮询规则选择服务 -》返回服务名称以供调用



Ribbion懒加载&饥饿加载

第一次加载耗时很长，然后将数据缓存

配置饥饿加载

```
ribbon:
 eager-load:
  enabled: true
  clients:
    -service1
    -service2
```

OpenFeign远程调用

Feign和RestTemplate一样，也是HTTP客户端请求工具，但是它的使用方式更加便捷。首先是依赖：

不同于RestTemplate,Feign是自动集成Ribbon的，同时默认不重试

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

feign配置部分：

自定义Feign的配置

Feign运行自定义配置来覆盖默认配置，可以修改的配置如下：

类型	作用	说明
feign.Logger.Level	修改日志级别	包含四种不同的级别：NONE、BASIC、HEADERS、FULL
feign.codec.Decoder	响应结果的解析器	http远程调用的结果做解析，例如解析json字符串为java对象
feign.codec.Encoder	请求参数编码	将请求参数编码，便于通过http请求发送
feign. Contract	支持的注解格式	默认是SpringMVC的注解
feign. Retryer	失败重试机制	请求失败的重试机制，默认是没有，不过会使用Ribbon的重试

添加接口：

```
@FeignClient("cloud_service")
public interface RoleClient
{
    //        List<Role> listRole=restTemplate.getForObject("http://localhost:8066/cache/role/list?key="+key,List.class);
    @GetMapping
```

```
    List<Role> getRoleCache(@RequestParam  String key);
}
```

性能优化解决方案

- 使用HTTP连接池
- 关闭日志

```
@FeignClient("cloud_service")
public interface RoleClient
{
    @GetMapping
    List<Role> getRoleCache(@RequestParam  String key);
}
```

Feign的最佳实践方式

- 继承方式

RPC到达controller，声明方式和controller必须一样，注意实际开发在Feign调用中可能会对请求头中加入相关的鉴权信息

一般不推荐继承方式，因为定义了同一接口，造成了紧耦合

- 抽取Feign

独立出feign-api，模块 负责实体，配置等服务

服务网关GateWay

统一网关GateWay(基于WebFlux实现，有响应式编程的优势)

> 服务网关是微服务架构中一个非常关键的角色，作为后端服务的统一入口，负责统筹和管理后端服务。能够提供的功能有路由功能、负载均衡、安全认证、流量染色、灰度发布、限流熔断、协议转换、日志收集等

基本作用：认证，校验，路由微服务请求，负载均衡，限流,路由过滤器

工作原理：

客户端向Spring Cloud Gateway发出请求。然后在Gateway Handler Mapping中找到与请求相匹配的路由，
将其发送到Gateway Web Handler。Gateway Web Handler再通过指定的过滤器链来将请求发送到我们实际的服务执行业务逻辑，然后返回。

配置方法：

```
<!-- 引入Gateway -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>

spring:
  cloud:
    gateway:
      routes:
        - id: auth-center
          uri: lb://auth-center
          predicates:
            - Path=/oauth
```

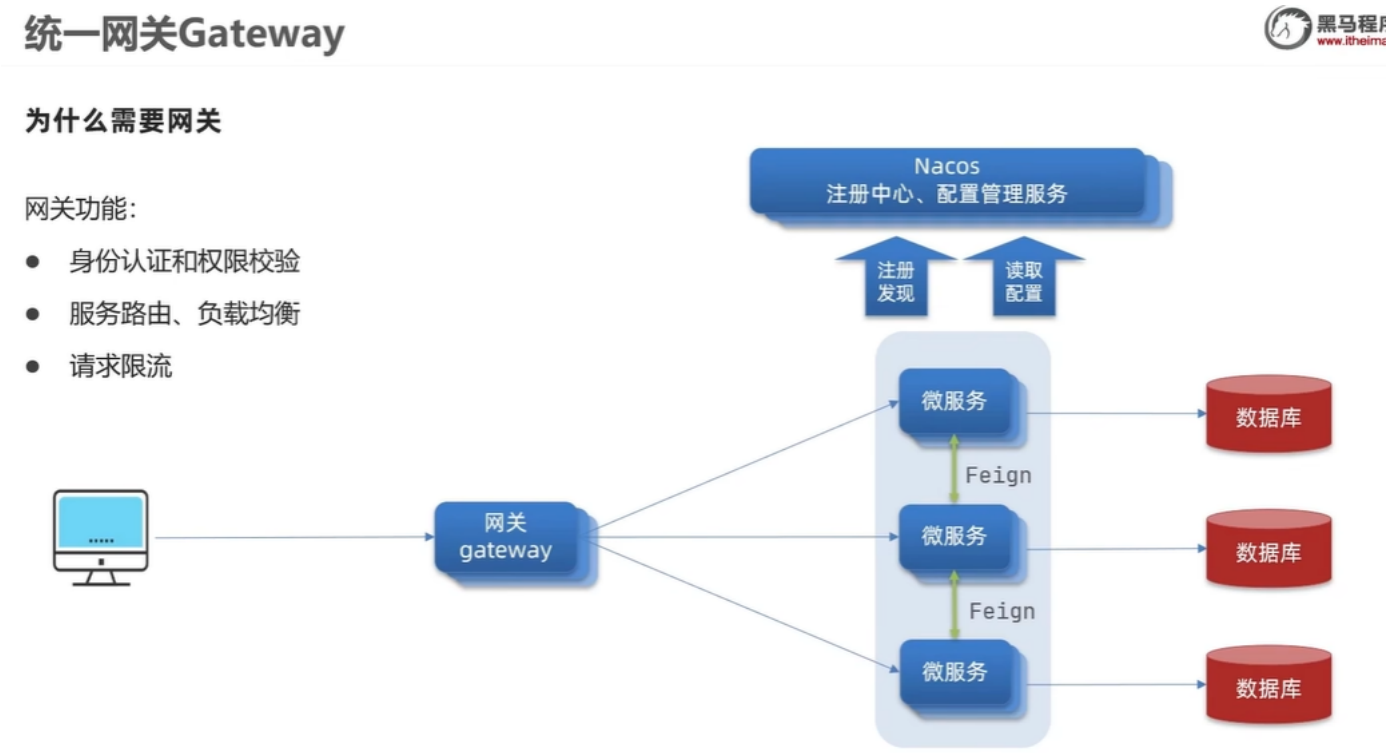
1. lb：固定格式，指的是从Nacos中按照名称获取微服务，并遵循负载均衡策略
2. service-name：Nacos注册中心的服务名称

断言：

Spring提供了11种基本的Predicate工厂：

名称	说明	示例
After	是某个时间点后的请求	- After=2037-01-20T17:42:47.789-07:00[America/Denver]
Before	是某个时间点之前的请求	- Before=2031-04-13T15:14:47.433+08:00[Asia/Shanghai]
Between	是某两个时间点之前的请求	- Between=2037-01-20T17:42:47.789-07:00[America/Denver], 2037-01-21T17:42:47.789-07:00[America/Denver]
Cookie	请求必须包含某些cookie	- Cookie=chocolate, ch.p
Header	请求必须包含某些header	- Header=X-Request-Id, \d+
Host	请求必须是访问某个host（域名）	- Host=**.somehost.org,**.anotherhost.org
Method	请求方式必须是指定方式	- Method=GET,POST
Path	请求路径必须符合指定规则	- Path=/red/{segment},/blue/**
Query	请求参数必须包含指定参数	- Query=name, Jack或者- Query=name
RemoteAddr	请求者的ip必须是指定范围	- RemoteAddr=192.168.1.1/24
Weight	权重处理	

简单架构：



搭建网关

适配器模式：将多个不同的接口实现顶级，变成希望的类或者属性

过滤器执行顺序：

过滤器执行顺序

- 每一个过滤器都必须指定一个int类型的order值，**order值越小，优先级越高，执行顺序越靠前。**
- GlobalFilter通过实现Ordered接口，或者添加@Order注解来指定order值，由我们自己指定
- 路由过滤器和defaultFilter的order由Spring指定，默认是按照声明顺序从1递增。
- 当过滤器的order值一样时，会按照 defaultFilter > 路由过滤器 > GlobalFilter的顺序执行。

可以参考下面几个类的源码来查看：

> `org.springframework.cloud.gateway.route.RouteDefinitionRouteLocator#getFilters()`方法是先加载`defaultFilters`，然后再加载某个`route`的`filters`，然后合并。
>
> `org.springframework.cloud.gateway.handler.FilteringWebHandler#handle()`方法会加载全局过滤器，与前面的过滤器合并后根据`order`排序，组织过滤器链

装饰者模式

网关跨域问题：

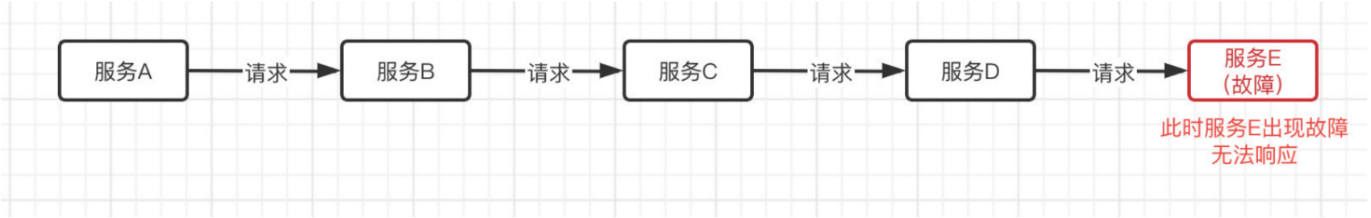
实际上就是RPC的codec层面

Hystrix服务熔断 @EnableHystrix

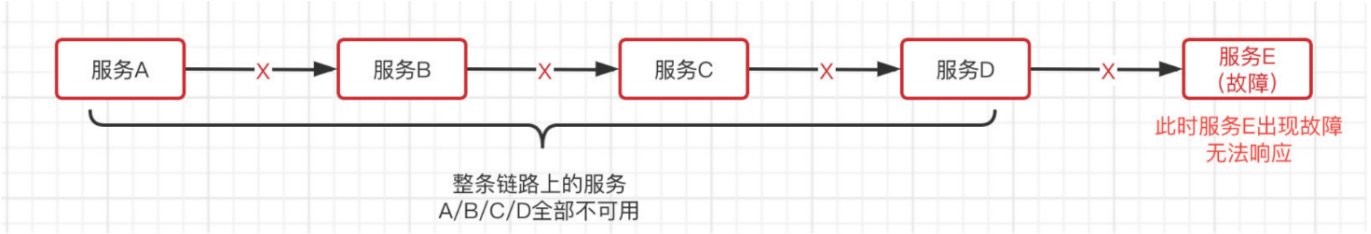
依赖：

```
<dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
                <version>2.2.10.RELEASE</version>
    </dependency>
```

我们知道，微服务之间是可以进行相互调用的，那么如果出现了下面的情况会导致什么问题？



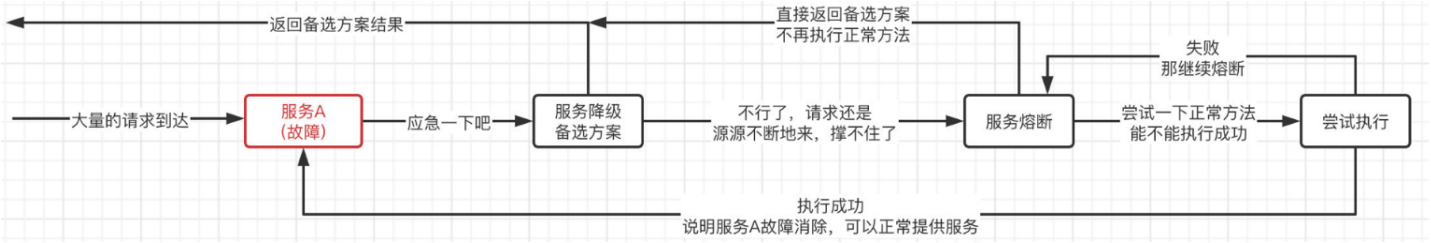
由于位于最底端的服务提供者E发生故障，那么此时会直接导致服务ABCD全线崩溃，就像雪崩了一样。



这种问题实际上是不可避免的，由于多种因素，比如网络卡顿、系统故障、硬件问题等，都存在一定可能，会导致这种极端的情况发生。因此，我们需要寻找一个应对这种极端情况的解决方案。

为了解决分布式系统的雪崩问题，SpringCloud提供了Hystrix熔断器组件，他就像我们家中的保险丝一样，当电流过载就会直接熔断，防止危险进一步发生，从而保证家庭用电安全。可以想象一下，如果整条链路上的服务已经全线崩溃，这时还在不断地有大量的请求到达，需要各个服务进行处理，肯定是会使得情况越来越糟糕的。

服务失败转为服务降级，而后当服务多次调用失败后，将会发生服务熔断



统一配置中心

作为远端配置统一管理，本地可以直接fetch然后使用，提高维护性

依赖：

```
<dependencies>
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-config</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-bootstrap</artifactId>
    </dependency>
</dependencies>
```